Task 02 — JavaScript Research

1. Differences between let, const, and var

• *var: Has function scope (or global scope if declared outside a function).*

• *let: Has block scope ({...}). It allows* reassignment *but does not allow* redeclaration *in the same scope.*

• *const: Has block scope and cannot be* reassigned *or re-declared.*

When to use:

• *Use const by default for variables that* shouldn't *be reassigned.*

• *Use let when you know the* variable's *value needs to change (e.g., loop counters).*

• *Avoid var in modern JavaScript to prevent unexpected scoping issues.*

Code Example :-

// var (Function Scope)

var x = 10;

var x = 20; // Allowed

console.log(x); // Output: 20

// let (Block Scope)

let y = 5;

// let y = 10; // SyntaxError: Identifier 'y' has already been declared

y = 15; // Allowed

console.log(y); // Output: 15

// const (Block Scope)

const z = 30;

// z = 40; // TypeError: Assignment to constant variable.

console.log(z); // Output: 30

2. Difference between == and === :-

• *== (Loose Equality): Compares two values for equality after performing type coercion (converting both*

values to a common type).

• === (Strict Equality): Compares both the value and the data type without performing type coercion. It returns true only if both operands are of the same type and value.

Code Example :-

```
const num = 10;

const strNum = &amp;quot;10&amp;quot;;

console.log(num == strNum); // Output: true (string &amp;quot;10&amp;quot; is coerced to number 10)

console.log(num === strNum); // Output: false (number !== string)
```

4. Difference between undefined and null

• undefined: Means a variable has been declared, but has not yet been assigned a value.

• null: Represents an intentional absence of any object value.

Code Example :-

```
let a;

console.log(a); // Output: undefined

console.log(typeof a); // Output: &amp;quot;undefined&amp;quot;

let b = null;

console.log(b); // Output: null

console.log(typeof b); // Output: &amp;quot;object&amp;quot;

console.log(a == b); // Output: true (both represent falsy/empty values)
```